TEXT CLASSIFICATION USING SUPPORT VECTOR MACHINE (SVM)

Python NLP Laboratory — Step-by-step execution and explanation

Objective: Implement an SVM-based text classifier using TF-IDF features. Evaluate it using accuracy, precision, recall and F1-score.

1. What is SVM?

SVM stands for Support Vector Machine. It is a supervised machine-learning algorithm used for classification and regression. In text classification, SVM learns to distinguish documents belonging to different categories, such as Sports and Technology.

Example documents: “India won the cricket match” belongs to Sports; “Artificial Intelligence is growing” belongs to Technology. The model learns from labelled examples and predicts the category of new documents.

2. How does SVM classify text?

SVM tries to identify a decision boundary that separates categories. Important terms are: **Hyperplane** — the decision boundary; **Support vectors** — the data points closest to the boundary; **Margin** — the distance between the boundary and the nearest data points; **Classification** — assigning a new document to a category.

3. Why use TF-IDF with SVM?

SVM cannot directly process sentences. TF-IDF converts text into numerical feature vectors. The SVM uses these numerical values to learn patterns and classify documents.

Workflow: Input text documents → text preprocessing → TF-IDF feature extraction → SVM classifier → predicted category (Sports or Technology).

4. Evaluation metrics

Accuracy measures the proportion of correctly classified documents. Precision measures how many documents predicted as positive actually belong to the positive category. Recall measures how many actual positive documents were correctly identified. F1-score is the harmonic mean of precision and recall.

Accuracy = (TP + TN) / (TP + TN + FP + FN)

Precision = TP / (TP + FP)

Recall = TP / (TP + FN)

F1-score = 2 × (Precision × Recall) / (Precision + Recall)

For this experiment, Technology is treated as the positive category. TP = Technology correctly predicted as Technology; TN = Sports correctly predicted as Sports; FP = Sports incorrectly predicted as Technology; FN = Technology incorrectly predicted as Sports.

5. Install the required library

Open Command Prompt or the VS Code terminal and run:

```
pip install scikit-learn
```

The program uses TfidfVectorizer, train_test_split, SVC, accuracy_score, precision_score, recall_score, f1_score, classification_report and confusion_matrix from Scikit-learn.

6. Complete Python program

Create a Python file named **svm_text_classification.py** and enter the following code:

```
# TEXT CLASSIFICATION USING SUPPORT VECTOR MACHINE

# STEP 1: Import required libraries
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix

# STEP 2: Create sample text documents
documents = [
    "India won the cricket match",
    "The football team scored a goal",
    "The player scored a century in cricket",
    "The team won the football tournament",
    "Cricket is a popular sport",
    "The coach trained the football players",
    "The batsman hit a six in the match",
    "The goalkeeper saved the penalty",
    "The tennis player won the match",
    "The basketball team scored points",
    "Artificial intelligence is transforming technology",
    "Machine learning is used in healthcare",
    "Python is a popular programming language",
    "Deep learning is a branch of artificial intelligence",
    "Computer networks connect different devices",
    "Data science uses machine learning algorithms",
    "Cloud computing provides online services",
    "Cybersecurity protects computer systems",
    "Robotics uses artificial intelligence",
    "Software development uses programming languages"
]

# STEP 3: Define category labels
labels = [
    "Sports", "Sports", "Sports", "Sports", "Sports",
    "Sports", "Sports", "Sports", "Sports", "Sports",
    "Technology", "Technology", "Technology",
    "Technology", "Technology", "Technology",
    "Technology", "Technology", "Technology",
    "Technology"
]

# STEP 4: Divide data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    documents, labels,
    test_size=0.25,
    random_state=42,
    stratify=labels
)

# STEP 5: Convert text into TF-IDF features
tfidf = TfidfVectorizer()
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

print("TF-IDF Feature Matrix:")
print(X_train_tfidf.toarray())

print("\nVocabulary:")
print(tfidf.get_feature_names_out())

# STEP 6: Create and train SVM classifier
svm_model = SVC(kernel='linear')
svm_model.fit(X_train_tfidf, y_train)

# STEP 7: Predict categories of test documents
y_pred = svm_model.predict(X_test_tfidf)

# STEP 8: Calculate evaluation metrics
accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(
    y_test, y_pred,
    pos_label="Technology",
    zero_division=0
)

recall = recall_score(
    y_test, y_pred,
    pos_label="Technology",
    zero_division=0
```

```
)

f1 = f1_score(
    y_test, y_pred,
    pos_label="Technology",
    zero_division=0
)

# STEP 9: Display evaluation results
print("\n===== SVM CLASSIFICATION RESULTS =====")
print("Accuracy:", round(accuracy, 4))
print("Precision:", round(precision, 4))
print("Recall:", round(recall, 4))
print("F1-Score:", round(f1, 4))

# STEP 10: Classification report
print("\nCLASSIFICATION REPORT:")
print(classification_report(
    y_test, y_pred,
    zero_division=0
))

# STEP 11: Confusion matrix
print("\nCONFUSION MATRIX:")
print(confusion_matrix(
    y_test, y_pred,
    labels=["Sports", "Technology"]
))

# STEP 12: Predict category of new documents
new_documents = [
    "Artificial intelligence helps computers learn",
    "The cricket team won the tournament",
    "Python programming is used in software development"
]

new_tfidf = tfidf.transform(new_documents)
predictions = svm_model.predict(new_tfidf)

print("\n===== NEW DOCUMENT PREDICTIONS =====")
for text, category in zip(new_documents, predictions):
    print("\nDocument:", text)
    print("Predicted Category:", category)
```

7. Explanation of the Python program

Step 1 — Import the libraries

TfidfVectorizer converts text into numerical features. SVC creates the Support Vector Machine classifier. The metric functions calculate accuracy, precision, recall and F1-score. classification_report displays a detailed report, while confusion_matrix shows correct and incorrect classifications.

Step 2 — Create sample text documents and labels

The program contains 20 labelled documents: 10 Sports documents and 10 Technology documents. Each document has a corresponding category label. The documents are the input features and the labels are the expected categories.

Step 3 — Divide the dataset

```
X_train, X_test, y_train, y_test = train_test_split(
    documents, labels,
    test_size=0.25,
    random_state=42,
    stratify=labels
)
```

test_size=0.25 reserves 25% for testing and 75% for training. With 20 documents, this gives 15 training documents and 5 testing documents. random_state=42 makes the split reproducible. stratify=labels maintains the category proportions in both sets.

Step 4 — Convert text into TF-IDF features

```
tfidf = TfidfVectorizer()
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)
```

fit_transform() learns the vocabulary from the training documents and converts them into TF-IDF vectors. transform() converts test documents using the same learned vocabulary. The test data is not used to fit the vectorizer.

Step 5 — Create and train the SVM classifier

```
svm_model = SVC(kernel='linear')
svm_model.fit(X_train_tfidf, y_train)
```

SVC() creates the SVM classifier. kernel='linear' specifies a linear decision boundary. fit() trains the model using the TF-IDF features and their known category labels.

Step 6 — Predict test categories

```
y_pred = svm_model.predict(X_test_tfidf)
```

The trained model predicts a category for each test document. These predictions are compared with the actual labels to evaluate the classifier.

Step 7 — Calculate evaluation metrics

accuracy_score(y_test, y_pred) calculates overall accuracy. precision_score(), recall_score() and f1_score() calculate the corresponding metrics for Technology, which is specified as the positive category using pos_label='Technology'. zero_division=0 prevents a division warning and returns zero if a metric has a zero denominator.

Accuracy = (number of correct predictions / total predictions) × 100. For example, 4 correct predictions out of 5 gives 80% accuracy.

Step 8 — Classification report

classification_report() displays precision, recall, F1-score and support for each category. Support is the number of actual test documents in that category. Macro average is the unweighted average across categories. Weighted average accounts for the number of documents in each category.

Step 9 — Confusion matrix

The confusion matrix is printed with the label order Sports, Technology. Rows represent actual categories and columns represent predicted categories.

Actual / Predicted	Sports	Technology
Sports	Correct Sports (TN)	Incorrect Technology (FP)
Technology	Incorrect Sports (FN)	Correct Technology (TP)

Step 10 — Predict new documents

Three new sentences are transformed using the TF-IDF vectorizer fitted on the training documents. The trained SVM predicts a category for each sentence. Expected semantic results are Technology for “Artificial intelligence helps computers learn”, Sports for “The cricket team won the tournament”, and Technology for “Python programming is used in software development”. Actual predictions depend on the small training sample and its learned vocabulary.

8. Expected output

The program displays the TF-IDF feature matrix and vocabulary, evaluation metrics, classification report, confusion matrix and predictions for new documents. The numerical metric values are calculated at runtime and are not fixed in advance.

```
===== SVM CLASSIFICATION RESULTS =====
Accuracy: [calculated value]
Precision: [calculated value]
Recall: [calculated value]
F1-Score: [calculated value]

CLASSIFICATION REPORT:
              precision  recall  f1-score  support
Sports
Technology

CONFUSION MATRIX:
[[... ...]
 [... ...]]

===== NEW DOCUMENT PREDICTIONS =====
```

```
Document: Artificial intelligence helps computers learn
Predicted Category: Technology

Document: The cricket team won the tournament
Predicted Category: Sports

Document: Python programming is used in software development
Predicted Category: Technology
```

9. How to execute the program

1. Open IDLE, VS Code, PyCharm or another Python environment.
2. Install Scikit-learn using `pip install scikit-learn`.
3. Create a Python file named **svm_text_classification.py**.
4. Paste the complete program and save it.
5. Open a terminal in the folder containing the file and run:

```
python svm_text_classification.py
```

Only Scikit-learn is required. The sample text documents are included in the script, so no external dataset download is needed.

10. Difference between Naive Bayes and SVM

Naive Bayes	SVM
Based on Bayes' theorem.	Based on decision boundaries and margins.
Uses conditional probabilities.	Finds a separating hyperplane.
Often fast for text classification.	Often effective with high-dimensional text features.
Can estimate class probabilities.	Standard SVC does not provide probabilities by default.

11. Applications of SVM-based text classification

• Email spam detection
• News article classification
• Sentiment analysis
• Fake news detection
• Customer review classification
• Document categorization
• Medical text classification

12. Student exercises

1. Add a third category called Politics.
2. Add 10 more documents to the dataset.
3. Change the SVM kernel from linear to rbf.
4. Compare SVM classification with Naive Bayes.
5. Enter five new sentences and predict their categories.
6. Compare precision, recall and F1-score for different datasets.

13. Quick knowledge check

Question 1: What does SVM stand for?
Answer: Support Vector Machine.

Question 2: Which technique converts text into numerical features in this experiment?
Answer: TF-IDF.

Question 3: What does the F1-score represent?
Answer: The harmonic mean of precision and recall.

Expected laboratory outcome: Students should be able to extract TF-IDF features, implement an SVM-based text classifier, classify unseen documents and evaluate classification performance using accuracy, precision, recall and F1-score.

Teaching suggestion: First demonstrate TF-IDF using two or three sample sentences. Then explain how SVM separates categories using a hyperplane. Finally, execute the complete program and explain the four evaluation metrics using the confusion matrix.

Note: The 20-document dataset is intended for learning the implementation, not for making reliable performance claims. For meaningful evaluation, use a larger, diverse and balanced dataset.